

基于机器视觉的物料搬运智能车的 控制程序设计

张兴华;肖仕驰

(广东白云学院,广州 510450)

摘要:传统的 AGV 小车依靠地面带有颜色的路线或者在地面埋电线进行导航。本文设计的小车依靠视觉模块非循迹直线导航,智能车根据距离的远近自动调节车速,小车自动找出最短路线行走,从而大幅度节省时间。文章给出了小车的控制算法程序。

Abstract: The traditional AGV car relies on the road with color on the ground or the navigation by buried wires on the ground. The car designed in this paper relies on the non-tracked linear navigation of the visual module. The intelligent car automatically adjusts the speed according to the distance, and the car automatically finds the shortest route to walk, thus greatly saving time. The control algorithm program of the car is given in this paper.

关键词: 视觉识别 无轨迹直线导航 控制程序

Key words: visual recognition ;trackless linear navigation ;control program

DOI:10.19475/j.cnki.issn1674-957x.2019.10.100

1 总体设计思路和方案

智能车视觉识别模块将外界信息处理并传输给主控板,主控板根据接收到消息,让电机驱动板控制小车靠近目标,让电机驱动板控制手臂手抓完成装卸的动作。

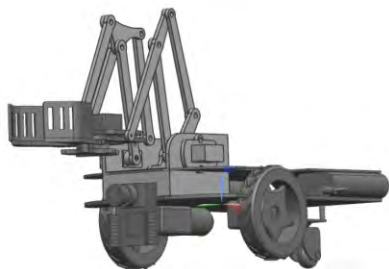


图 1 小车的设计图

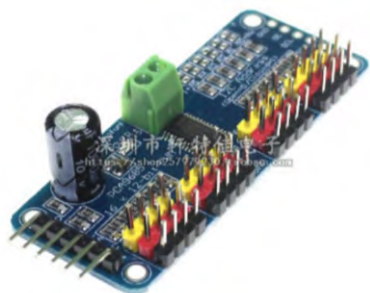


图 2 16 路舵机驱动板 PCA9685

小车使用 OPENMV 视觉模块实现对目标物体的追踪与距离检测,使用 THB6612FNG 电机驱动模块的 pwma 和 pwmb 分别接 OPENMV 的 P6 和 P9 控制两路直流电机 PWM 引脚上的电压值,使用 16 路舵机驱动板 PCA9685 通过板载的 I2C 驱动三路舵机的机械臂,实现目标物体的装载。

基金项目: 广东白云学院 2018 年度校级项目 (项目编号: 2018BYKYK11)。

作者简介: 张兴华 (1967-),男,广东白云学院讲师,工程师,研究方向为智能装备与机器人。

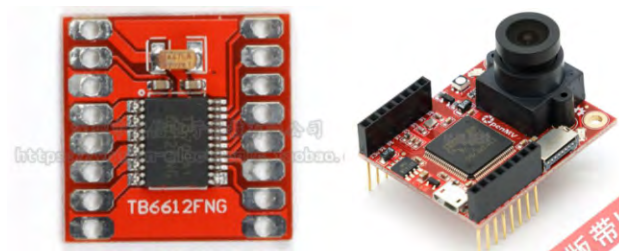


图 3 THB6612FNG 电机驱动模块 图 4 OPENMV 视觉模块

2 控制程序设计

应用 Python 语言编写控制程序 main.py 如下:

```
import sensor, image, time
import car #car.py
from pid import PID #pid.py
import time,pyb
from servo import Servos #servo.py
from machine import I2C, Pin
i2c = I2C(sda=Pin('P5'), scl=Pin('P4'))
servo = Servos (i2c, address=0x40, freq=40, min_us=
500, max_us=2500, degrees=180)
sensor.reset()# 摄像头复位
sensor.set_pixformat(sensor.RGB565)
# 摄像头传感器设置为 16 位色
sensor.set_framesize(sensor.QQVGA)
# 摄像头模式
sensor.skip_frames(10)
# 跳过 10 帧图像
sensor.set_auto_whitebal(False)
# 不使用自动白平衡
clock = time.clock()
red_threshold =(68, 9, 17, 89, -37, 73)
# 设定红色目标乒乓球的阈值
black_threshold =(25, 0, 81, -63, 106, -97)# 设定黑色
```

围墙的阈值

```
green_threshold =(65, 18, -21, -64, 68, -30)# 设定绿色
```

色仓库墙壁的阈值

```
red_size_threshold = 1200 # 红色目标的尺寸阈值
```

```
black_size_threshold = 10000 # 黑色目标的尺寸阈值
```

```
green_size_threshold =18000# 绿色目标的尺寸阈值
```

```
x_pid_r = PID(p=1.50, i=0.20, d=0.002,imax=5) #red 目
```

标左右距离的 PID 调节

```
h_pid_r = PID(p=0.003, i=0.020,d=0.008, imax=45) #
```

红色目标前后距离的 PID 参数值

```
x_pid_b = PID(p=0.40, i=0.10, d=0.004,imax=5) #
```

黑色围墙的 PID 参数值

```
h_pid_g = PID(p=0.0005, i=0.100,d=0.008, imax=80)
```

绿色仓库墙壁的 PID 参数值

```
def find_max_b(blob_black):
```

```
# 寻找最大黑色目标
```

```
global max_black# 黑色目标
```

```
max_size_b=0
```

```
for blob in blob_black:
```

```
if blob[2]*blob[3] > max_size_b:# 如果目标的宽
```

度 * 高度大于最大值

```
max_black=blob
```

```
max_size_b=lob[2]*blob[3]
```

```
return max_black
```

```
def find_max_r(blob_red):# 找红色目标
```

```
global max_red
```

```
max_size_r=0
```

```
for blob in blob_red:
```

```
If blob[2]*blob[3]> max_size_r:
```

```
max_red=blob
```

```
max_size_r=blob[2]*blob[3]
```

```
return max_red
```

```
def find_max_g(blob_green):
```

```
global max_green
```

```
max_size_g=0
```

```
for blob in blob_green:
```

```
If lob[2]*blob[3] >max_size_g:
```

```
max_green=blob
```

```
max_size_g=blob[2]*blob[3]
```

```
return max_green
```

```
def servo_clip():# 机械臂手爪抓紧
```

```
servo.position(0,0) # 大臂复位
```

```
servo.position(1,120) # 小臂复位
```

```
servo.position(2,100) # 手爪复位
```

```
time.sleep(500)
```

```
servo.position(0,100) # 大臂向前
```

```
servo.position(1,40) # 小臂向后
```

```
time.sleep(500)
```

```
car.run(30,30)
```

```
time.sleep(600)
```

```
servo.position(2,180) # 手爪合拢
```

```
time.sleep(500)
```

```
servo.position(0,0) # 大臂复位
```

```
servo.position(1,120) # 小臂复位
```

```
def servo_clip_r():# 机械臂手爪复位
```

```
servo.position(0,0) # 大臂复位
```

```
servo.position(1,120) # 小臂复位
```

```
servo.position(2,100) # 手爪复位
```

```
time.sleep(500)
```

```
servo.position(0,100) # 大臂向前
```

```
servo.position(1,40) # 小臂向后
```

```
car.run(30,30)
```

```
time.sleep(200)
```

```
servo.position(2,180) # 手爪合拢
```

```
time.sleep(500)
```

```
servo.position(0,0) # 大臂复位
```

```
servo.position(1,120) # 小臂复位
```

```
def servo_put():# 机械臂手爪松开
```

```
servo.position(2,110) # 手爪松开
```

```
time.sleep(400)
```

```
car.run(50,50)
```

```
time.sleep(1000)
```

```
global temp
```

```
temp=0
```

```
while(True):
```

```
clock.tick()
```

```
img = sensor.snapshot()
```

```
blob_black = img.find_blobs([black_threshold])
```

```
max_black = find_max_b(blob_black)
```

```
img.draw_rectangle(max_black[0:4])
```

```
img.draw_cross(max_black[5], max_black[6])
```

```
h_error_black = max_black [2]*max_black [3] -
```

```
black_size_threshold
```

```
if h_error_black>5000:
```

```
car.run(-50,50)
```

```
car.run(-50,50)
```

```
elif temp==0:
```

```
blob_red = img.find_blobs([red_threshold])
```

```
if blob_red:
```

```
max_red=find_max_r(blob_red)
```

```
img.draw_rectangle(max_red[0:4])
```

```
img.draw_cross(max_red[5], max_red[6])
```

```
#blob[5]是 x 坐标 blob[6]是 y 坐标
```

```
x_error = max_red[5]-img.width()/2
```

```
h_error= max_red[2]*max_red[3]-red_size_threshold
```

```
x_output=x_pid_r.get_pid(x_error,1)
```

```
h_output=h_pid_r.get_pid(h_error,1)
```

```
a=-h_output*1.4+x_output/2.2
```

```
b=-h_output*1.4-x_output/2.2
```

```
print("a: ", a)
```

```
print("b: ", b)
```

```
print("h_error:", h_error)
```

```

car.run(a,b)
if (-650<h_error<-450):
    car.run(0,0)
    if max_red[2]/max_red[3]>1.2:
        servo_clip_r()
        car.run(-30,30)
        time.sleep(500)
        car.run(0,0)
        time.sleep(200)
        temp=1
    else:
        servo_clip()
        car.run(-30,30)
        time.sleep(500)
        car.run(0,0)
        time.sleep(200)
        temp=1
else:
    car.run(-30,30)
else:
    blob_green = img.find_blobs([green_threshold])
if blob_green:
    max_green=find_max_g(blob_green)
    img.draw_rectangle(max_green[0:4])
    img.draw_cross(max_green[5], max_green[6])
    x_error_g= max_green[5]-img.width()/2
    h_error_g = max_green [2]*max_green [3] -
green_size_threshold
    x_output_g=x_pid_g.get_pid(x_error_g,1)
    h_output_g=h_pid_g.get_pid(h_error_g,1)
    c=-h_output_g/1.2+x_output_g/2
    d=-h_output_g/1.2-x_output_g/2
    print("c: ",c)
    print("d: ",d)
    car.run(c,d)
    if (850<h_error_g<950):
        car.run(40,40)
        time.sleep(200)
        car.run(0,0)
        servo_put()
        temp=0
        car.run(-100,-100)
        time.sleep(400)
        car.run(-60,60)
        time.sleep(500)
        car.run(0,0)
        time.sleep(200)
    else:
        car.run(10,40)
#Car.py
from pyb import Pin, Timer

```



图5 智能车准备抓取红色目标

```

inverse_left=True
inverse_right=False
ain1 = Pin('P0', Pin.OUT_PP)
ain2 = Pin('P1', Pin.OUT_PP)
bin1 = Pin('P2', Pin.OUT_PP)
bin2 = Pin('P3', Pin.OUT_PP)
ain1.low()
ain2.low()
bin1.low()
bin2.low()
pwma = Pin('P6')
pwmb = Pin('P9')
tim2 = Timer(2, freq=1000)
tim4 = Timer(4, freq=1000)
ch1 = tim2.channel(1, Timer.PWM, pin=pwma)
ch2 = tim4.channel(3, Timer.PWM, pin=pwmb)
ch1.pulse_width_percent(0)
ch2.pulse_width_percent(0)
def run(left_speed, right_speed):
    if inverse_left==True:
        left_speed=(-left_speed)
    if inverse_right==True:
        right_speed=(-right_speed)
    if left_speed < 0:
        ain1.low()
        ain2.high()
    else:
        ain1.high()
        ain2.low()
    ch1.pulse_width_percent(abs
(left_speed))

```



图6

基于红外光传感器的铁路货车闸瓦磨耗监测方法研究

申燕飞^①; 张瑞国^②; 秦勇^③

(^①眉山中车制动科技股份有限公司, 眉山 620010 ;

^②中车眉山车辆有限公司, 眉山 620010 ;^③西南交通大学, 成都 611756)

摘要: 闸瓦是铁路货车制动系统中的重要部件,但目前国内对货车闸瓦的监测仍然停留在人工检测层面,其结果可靠性较低,且只能在货车到站时静态检测,因此本文提出了利用红外光传感器采集一辆车上的 8 个闸瓦磨耗数据后整理生成一帧完整的数据并以无线方式发送至上位机系统。根据闸瓦磨耗值判断出闸瓦的工作状态,并发出预警信息,实现闸瓦磨耗的自动化监测。

关键词: 闸瓦磨耗监测 数据采集 单片机 红外光传感器

DOI:10.19475/j.cnki.issn1674-957x.2019.10.101

0 引言

闸瓦制动方式是中国铁路货车主要的制动方式。由于闸瓦制动的特点,闸瓦磨耗量较大且磨损速度也较快。而随着我国经济的飞速发展,铁路货车正朝着“高速”和“重载”方向发展,闸瓦的磨损问题也将日益严重。但国内仅仅实现了对出厂闸瓦的超声波检测,对在役闸瓦仍然采用人工检测,其结果可靠性较低,且只能在货车到站静态时检测,货车运行过程中闸瓦的磨耗情况将一无所知。故对在役闸瓦的自动化监测是必然趋势。

1 铁路货车闸瓦磨耗的自动化监测系统总体设计

本论文提出铁路货车闸瓦磨耗的自动化监测方法,总体框架如图 1 所示,主要包括数据采集系统、数据传输系统和数据处理系统。

数据采集系统将红外光传感器安装在货车闸瓦踏面的对侧。在货车非制动的状态下,用测得的传感器与闸瓦踏面之间的距离值减去传感器与最新的闸瓦踏面之间的距离值得到闸瓦磨耗数据。然后传输至单片机处理,生成一帧完整的数据。然后将这帧数据以无线的方式发送至上

位机系统。

数据处理系统以单片机为核心处理器,提取出传感器与闸瓦之间的距离值。然后依据闸瓦磨耗的数据特征,判断出闸瓦的工作状态。当列车处于非制动状态时,可以正常测得闸瓦磨耗值。当列车处于制动状态时,无法测得闸瓦磨耗值,将该数据剔除。但是可以根据测得的数据判断出闸瓦处于工作抱闸状态还是非工作抱闸状态。最后针对不同的闸瓦状态,发出相应的指令至显示屏,实时显示磨耗数据以及闸瓦状态。同时当闸瓦磨耗超出安全限度以及非工作抱闸状态时及时触发报警模块,发出预警信息。

无线传输系统以无线 WIFI 模块为核心器件,单片机通过 UART 的方式将数据传输至无线发射模块,通过无线方式传送至上位机无线接收器。

2 数据采集系统

2.1 红外光传感器

货车闸瓦的安装部位比较复杂,当机车制动时,闸瓦紧紧地按压车轮踏面,故测量货车闸瓦的磨耗,只能用非接触的方法测量。非接触的测距传感器主要有三种 超声

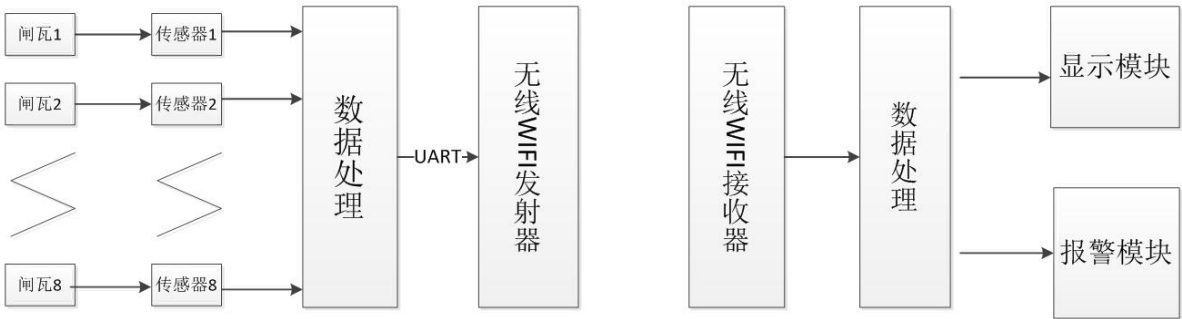


图 1 系统总体框图

```
if right_speed < 0:
    bin1.low()
    bin2.high()
else:
    bin1.high()
    bin2.low()
ch2.pulse_width_percent(abs(right_speed))
```

3 结论

本文中给出的 Python 控制程序可以很精准地实现小车的工作要求。

参考文献:

- [1] 韩九强. 机器视觉技术及应用[M]. 高等教育出版社, 2012.
- [2] 陈兵旗. 机器视觉技术及应用实例详解[M]. 化学工业出版社, 2014.